Deep Learning Model from Scratch

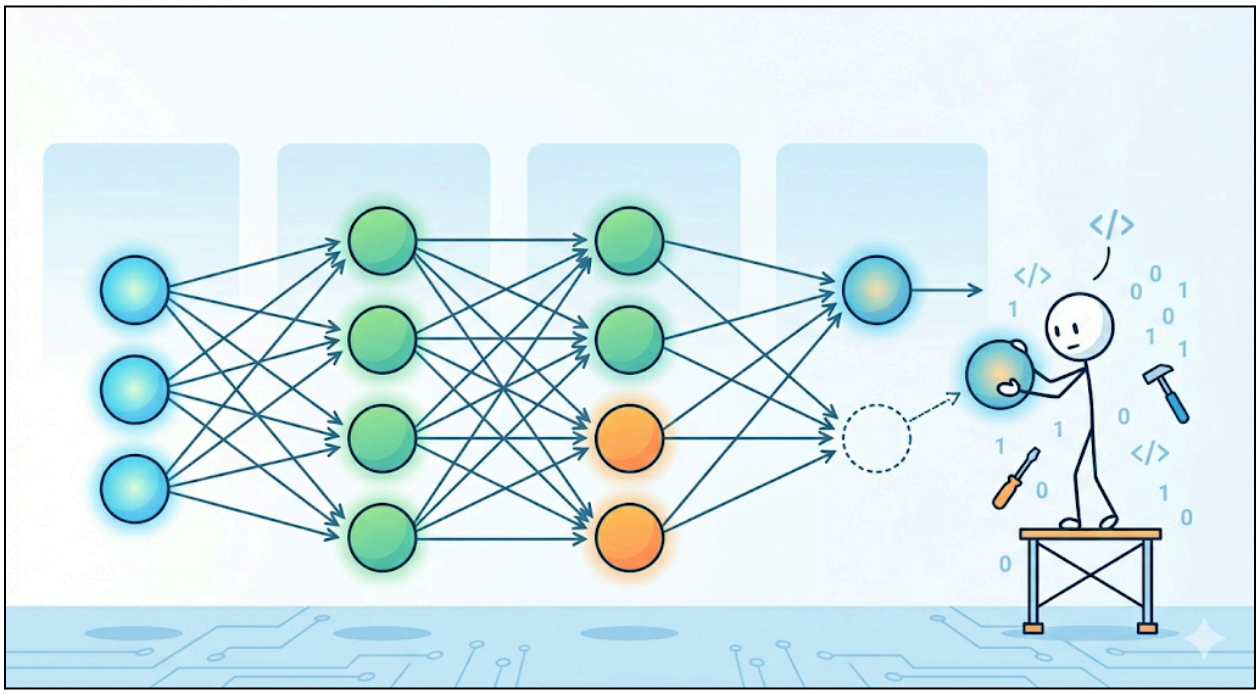

Figure T.0 Neural Network Construction [1]

0. Project Overview

Curious about the ongoing 'AI Revolution,' we sought out to develop a neural network from first principles. We trained our model to recognize handwritten numbers 0-9 from the MNIST image database. For a conceptual introduction to deep learning, we consulted the visual explanations presented by Grant Sanderson [2]. This, and a study of Michael Nielsen's textbook [3], gave us sufficient background to begin the development of our model. Nielsen also published examples to accompany his work [4], including a Python model which we used as a reference to validate the serial version of our code. In order to apply high-performance programming concepts, we developed our code in C.

Through the use of memory and compiler optimizations, manual vectorization, and multicore processing, we were able to reduce the overall runtime by a factor of **156X** while maintaining a **96.57%** identification accuracy.

Developing Serial Baseline Algorithm

Before applying high-performance programming techniques, we needed to establish a serial baseline algorithm as our benchmark. Prioritizing correctness over computational speed, we designed this implementation to mirror the underlying mathematical formulas exactly. Each function processes a single image at a time through feedforward, backpropagation, and gradient descent. Also, the shuffling of training data in between epochs is consistently random between runs, such that it produces identical parameters and therefore results in the testing phase. This means we can verify future optimizations do not compromise the algorithm's integrity.

Snippet 0.A Pseudo-code of Algorithm Architecture

```
//TRAINING
for epoch = 1 .. E //10
    shuffle(train_data_indices) //seeded... 60,000 samples

    for each batch of size B //100
        zero(gradient_accumulators)

        for each sample in batch
            load sample into input buffer
            feedforward()
            backpropagation(label)
            accumulate_gradients()
        end

        update_parameters()
    end
end

//TESTING
correct = 0
for each sample in test_data
    load sample into input buffer
    feedforward()
    prediction = argmax(output_activations)

    if prediction == label
        correct += 1
end

print(correct / # of test_data) //10,000
```

The result is an artificial neural network that uses feedforward and is trained using backpropagation with mini-batch stochastic gradient descent. We chose to use float types because they provided a sufficient degree of specificity, and operations are faster than double would be. The model takes a 28×28 grayscale image as input, where each value corresponds to the brightness of a single pixel. It has a total of four dense neuron layers: the first is the flattened MNIST input (784 neurons), followed by a hidden layer of 100 neurons, a second hidden layer of 16 neurons, and an output layer of 10 neurons corresponding to digits 0-9. Sigmoid activation is used between layers, producing normalized activations on a 0–1 scale, and the output neuron with the highest activation determines the predicted digit.

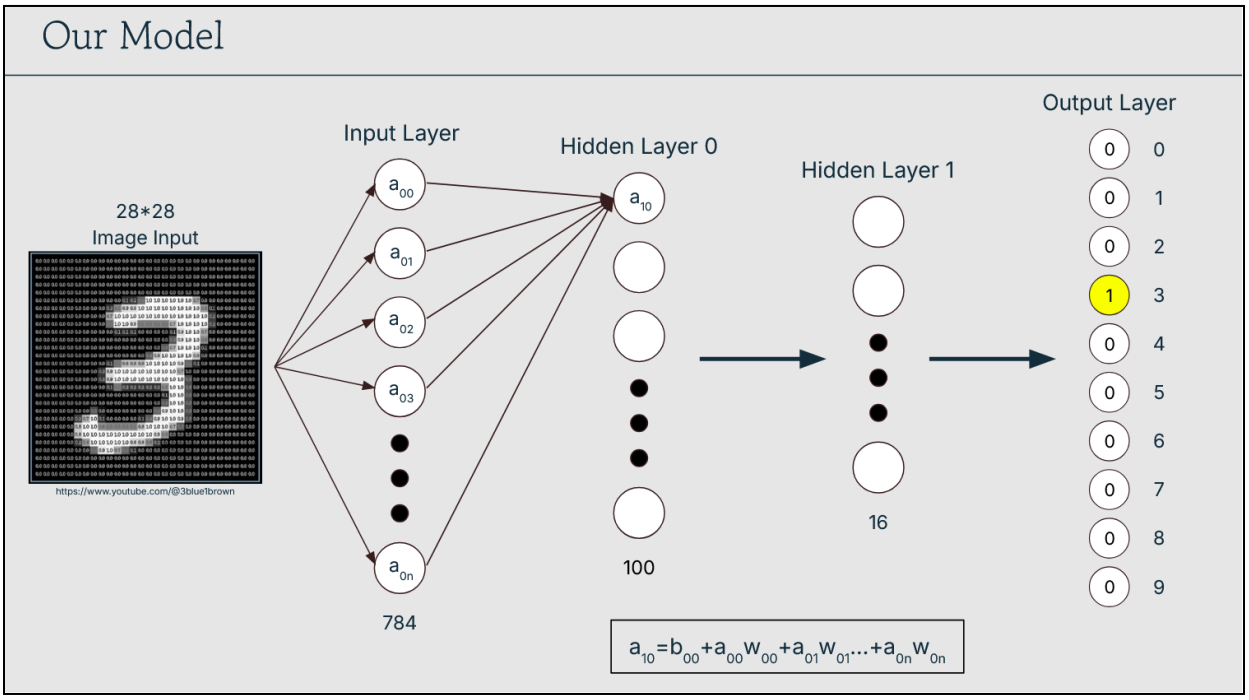

Figure 0.0 Model Visualization

Results

We randomize the training data to be reused over 10 epochs, and each batch within an epoch contains 100 samples. As we continued tuning hyperparameters, we found that a learning rate of 3 provided the highest accuracy. This is extremely large in the context of modern neural networks, but fits well with the scale of our model. The serial baseline algorithm ran in **195.1802 seconds** with a classification accuracy of **96.57%**.

Identifying Optimization Hotspots

Amdahl's Law states that the maximum speedup from optimizing a part of a program is limited by the fraction of total execution time spent in that part. To most efficiently optimize our program, we must therefore search for the particular functions that make up most of the execution time. We do this by explicitly measuring functions that we suspect to be hotspots. Based on the hyperparameters of our model, this is likely to be any processing steps between the input layer and the first hidden layer. For this, we used clock_gettime() from the native time library in C. Using CLOCK_MONOTONIC meant that we were immune to wall-clock adjustments and were able to achieve nanosecond resolution. There was some overhead for each call (on the order of tens of nanoseconds), but we can see in the runtime prints of our serial code that the average time spent executing each of the examined functions far outweighs this cost.

Snippet 0.B Terminal output of Serial Baseline Code with Kernel-Specific timing

```
./mnist_c
init_MNIST:  0.0004 s
  kernel_matrix_vector_mult: total=70.1496 s  calls=600000  avg=116915.94 ns
  kernel_vector_vector_mult: total=50.1674 s  calls=600000  avg=83612.42 ns
  kernel_matrix_matrix_add:  total=61.8990 s  calls=600000  avg=103165.01 ns
train_MNIST: 194.0044 s
Test accuracy: 9657 / 10000 (96.57%)
test_MNIST:  1.1754 s
Total: 195.1802 s
```

Note that when timing initialization, training, and testing, training takes almost all of the time. Within training, **our three identified kernel functions account for >90% of total time**.

With a bit of architectural refactoring, we isolated these three functions into their own kernel.c/h files as distinct kernel_… function calls so that we could apply targeted optimizations and perform precise benchmarking. Below are the characteristics of each. **These three kernels account for >90% of memory accesses and FLOPs in the program**. We can see by the arithmetic intensity that our optimizations will be limited, as we are certainly memory-bound.

Kernel	Role	Time Complexity	Arithmetic Intensity
`kernel_matrix_vector_mult`	Computes the pre-activation z = `W·a` for each layer during the forward pass.	n output elements × I multiply-adds = **O(n·I)**	2·n·I FLOPs / (n·I + I + n) × 4 B ≈ 2nI / 4nI = **0.5 FLOP/B**
`kernel_vector_vector_mult`	Computes the weight gradient `∂C/∂W = δ ⊗ a` as an outer product during backpropagation.	n rows × I columns, 1 multiply each = **O(n·I)**	n·I FLOPs / (n + I + n·I) × 4 B ≈ nI / 4nI = **0.25 FLOP/B**
`kernel_matrix_matrix_add`	Accumulates per-sample weight gradients into a batch sum `G_sum += G_sample`.	n·I independent element-wise additions = **O(n·I)**	n·I FLOPs / (3·n·I) × 4 B = 1 / 12 ≈ **0.083 FLOP/B**

Figure 0.1 Kernel Code- Note: n = # of output neurons, I = # of input features

1. Serial Optimizations

To start our optimization journey, we implemented several serial optimizations for our model. All of these optimizations were made with the -O0 compiler flag set.

Removing Memory Aliasing

One important compiler blocker is Memory Aliasing. When memory addresses are determined at runtime, the compiler does not know whether two or more reads/writes are accessing the same memory locations or not. To stay on the safe side, the compiler assumes that they are, and writes to and reads from memory every time a location is accessed rather than using a temporary register.

Let's look at two examples of this in our project. In Snippet 1.A, v_out_loc[i] in the innermost loop does not depend on j. Thus, the compiler should be able to hoist the memory reference out of the loop and replace it with a temp accumulator. However, it does not do this due to Memory Aliasing. To fix this, we manually hoisted the memory reference out of the loop and used a temporary accumulator. Snippet 1.B is similar. v1_start[i] in the innermost loop does not change as j changes, and can be hoisted out of the loop.

Snippet 1.A Code containing a memory aliasing hazard in matrix_vector_mult

```
for (int i = 0; i < rows; i++) {
  v_out_loc[i] = 0;
  for (int j = 0; j < cols; j++) {
    v_out_loc[i] += weights[i*cols + j] * lastLayerActivations[j];
  }
}
```

Snippet 1.B Code containing a memory aliasing hazard in vector_vector_mult

```
for (int i = 0; i < v1len; i++) {
  for (int j = 0; j < v2len; j++) {
    vout_start[i*voutcol + j] = v1_start[i] * v2_start[j];
  }
}
```

In Figure 1.0, we can see the effect of these changes. vector_vector_multiply sees a 4000ns drop in time, while matrix_vector_multiply does not show as dramatic of a decrease.

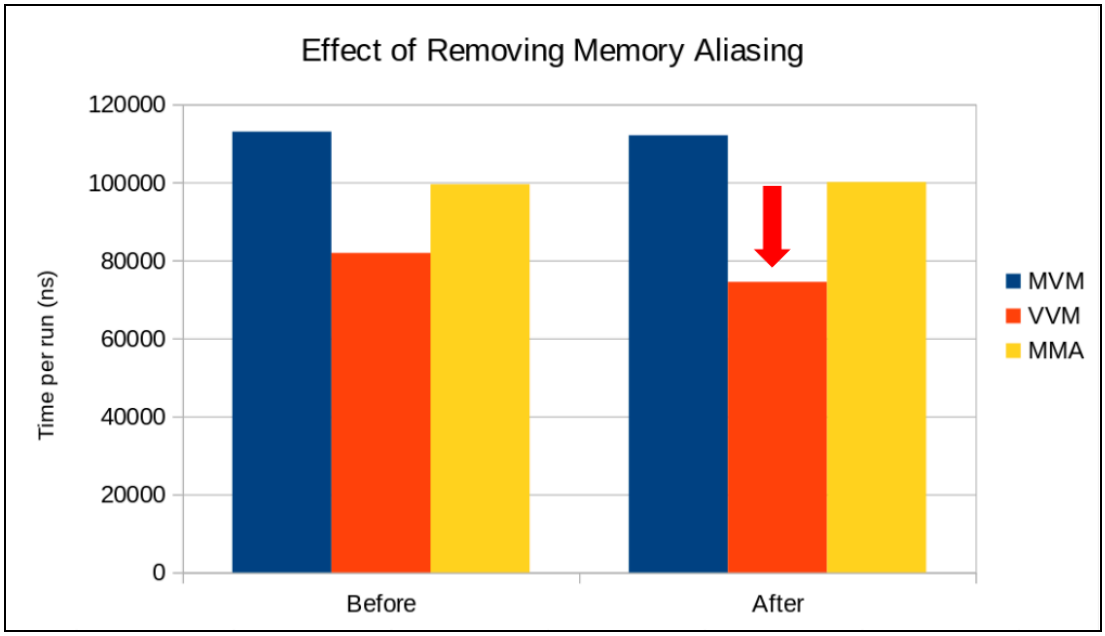

Figure 1.0 Effect of Removing Memory Aliasing

Loop Unrolling

A classic and highly effective serial optimization, loop unrolling was a significant improvement to our serial code. In Snippet 1.C, we unrolled the inner loop of MVM, using 6 accumulators. In this case, this not only reduces loop overhead, but also serves to fill up the load, multiply, and add pipelined functional units in the CPU. For the other two functions, VVM and MMA, there was no accumulation, so loop unrolling only reduced loop overhead.

Snippet 1.C Code exhibiting loop unrolling in matrix_vector_mult

```
for (i = 0; i < rows; i++) {
  row_offset = i*cols;

  for (j = 0; j < cols - 5; j+=6) {
    offset = row_offset + j;
    sum0 += weights[offset] * lastLayerActivations[j];
    sum1 += weights[offset + 1] * lastLayerActivations[j + 1];
    sum2 += weights[offset + 2] * lastLayerActivations[j + 2];
    sum3 += weights[offset + 3] * lastLayerActivations[j + 3];
    sum4 += weights[offset + 4] * lastLayerActivations[j + 4];
    sum5 += weights[offset + 5] * lastLayerActivations[j + 5];
  }

  // finish remaining elements
  for (; j < cols; j++) {
   sum0 += weights[row_offset + j] * lastLayerActivations[j];
```

```
  }

  v_out_loc[i] = sum0 + sum1 + sum2 + sum3 + sum4 + sum5;
  sum0 = 0;
  sum1 = 0;
  sum2 = 0;
  sum3 = 0;
  sum4 = 0;
  sum5 = 0;
}
```

As seen in Figure 1.1, all functions benefited from loop unrolling, with a factor of 6 unrolls giving the greatest performance. As expected, MVM had the biggest improvement due to the pipeline loading, with nearly a 50% reduction in time. VVM and MMA also benefited from reduced loop overhead due to reduced branch prediction frequency.

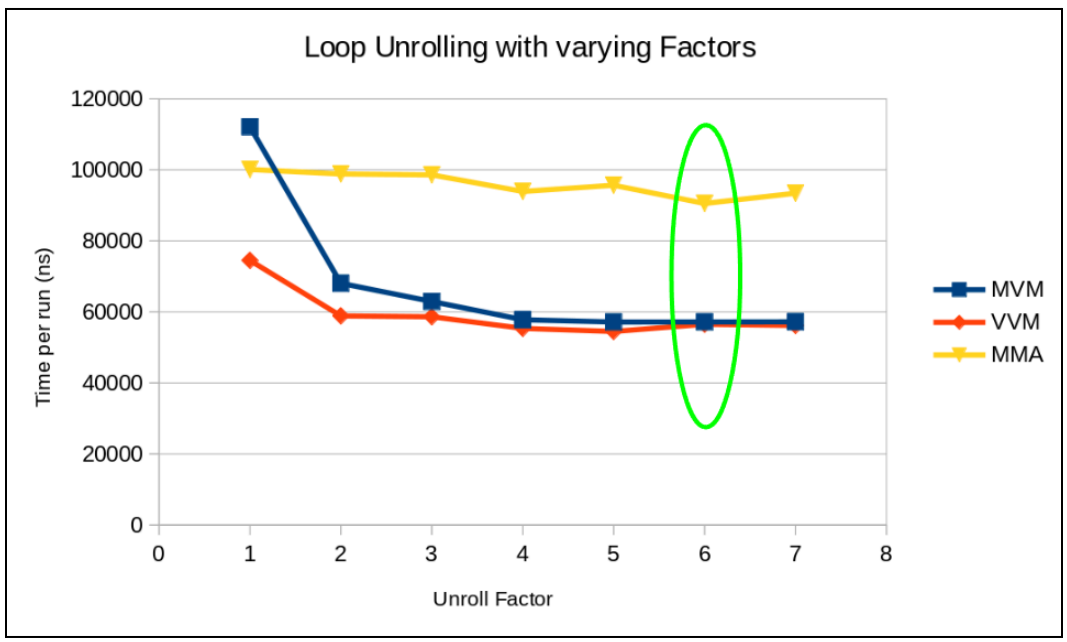

Figure 1.1 Effect of Loop Unrolling with varying factors

Common Subexpression Elimination

Another optimization to make the compiler's life easier: eliminating common subexpressions. Comparing Snippet 1.A and 1.C, we can see the row_offset and offset variables now replace what would be the repeated expression i*cols and row_offset + j, respectively. Without these separate variables, the processor would recompute the subexpression i*cols + j for every value of j. This computation only needs to be done once, so computing it and moving it into a single variable guides the compiler to store the value in a register for much faster access. In Figure 1.2,

we can see the effect of eliminating these subexpressions. It yields improvements for all functions, especially MMA, with a time decrease of 36%.

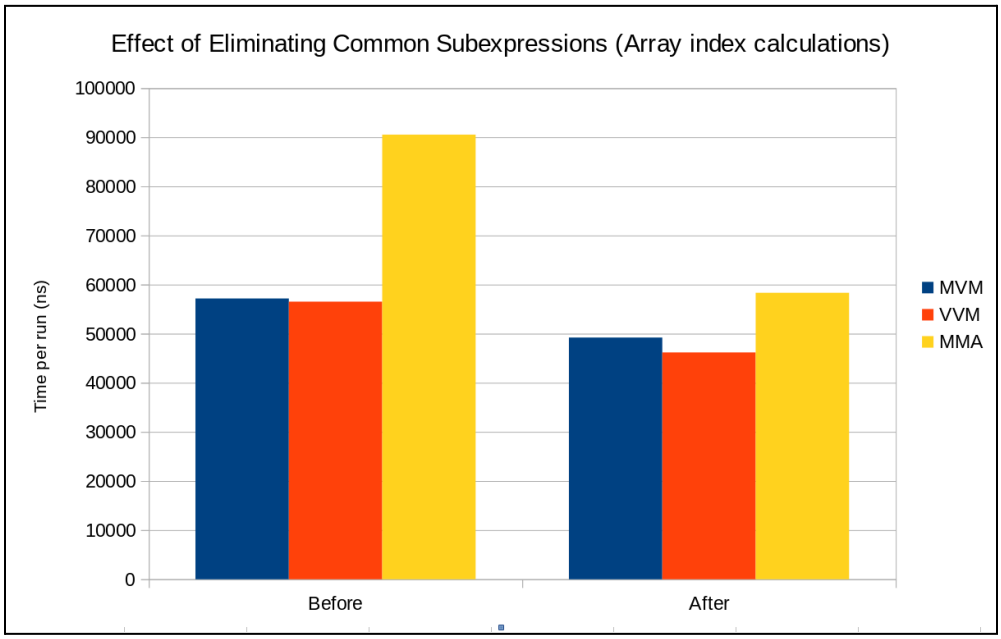

Figure 1.2 Effect of Eliminating Common Subexpressions

Compiler Optimizations

Finally, we let the compiler try out whatever remaining optimizations it might have wanted to do, and set the optimization flag to -O1, -O2, and -O3. See Figure 1.3 for the results.

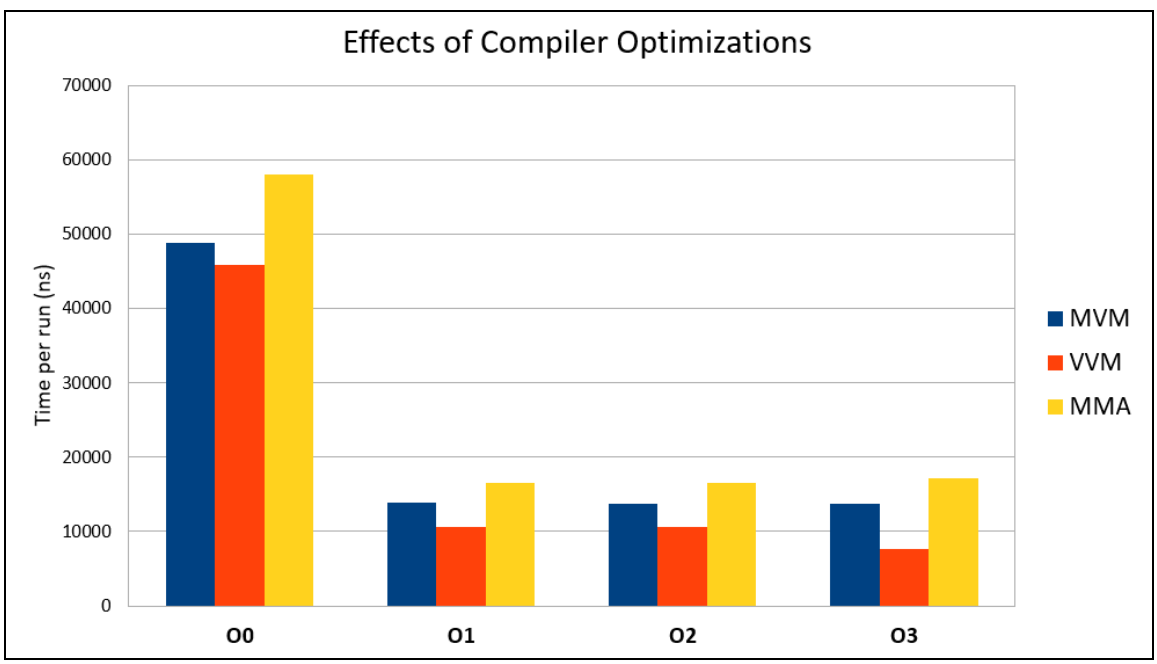

Figure 1.3 Effects of Compiler Optimizations

2. Vector Optimizations

Now we foray into parallel land. Intel's SSE and AVX extensions provide powerful parallel computation units for processing multiple numbers simultaneously. In our code, we use the AVX extensions, which use 256-bit registers, and the float data type. Each AVX register can hold 8 floats, meaning we can use them to process up to 8 matrix/vector entries at a time. In this section, all functions are run with the -O1 optimization flag, except for the first Basic AVX run in Figure 2.0. We also ran the functions with -O2 and -O3, but there was negligible speedup.

Basic AVX Optimizations

In our first round of optimizations, we opted to use only basic AVX functions. These were add, multiply, broadcast, and reduce. As we can see in Figure 2.0, the basic AVX functions are a huge improvement over the original serial baseline (see Figure 1.0). However, with -O0 optimization, they do not beat the -O3 optimized serial code, but as soon as we turn -O1 optimization on, they do significantly better than serial. The substantial difference between -O0 and -O1 optimizations for vectors can be attributed to the stack backup that the compiler makes in -O0 (debug) mode. Because vectors are such large registers, the memory latency accrued from loading and storing a vector register variable from and to the stack is significant.

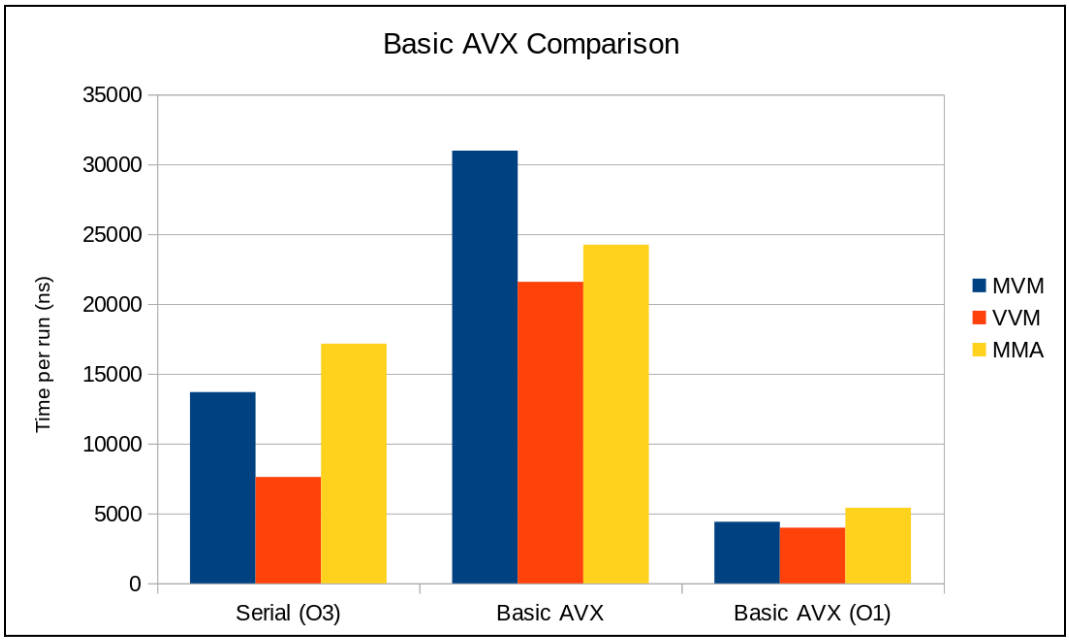

Figure 2.0 Effect of Basic AVX Optimizations

Loop Unrolling, Pt. 2

As with our serial code, we loop unrolled our vector code as well. Vector functional units are pipelined just like serial functional units, and unrolling reduces loop overhead nonetheless, so we tried several unroll factors to see what performance gains they might have to offer.

Snippet 2.A Code exhibiting vector loop unrolling in vector_vector_mult

```
for (j = 0; j < v2len - (AVX_STRIDE_6 - 1); j += AVX_STRIDE_6) {

    __m256 v2_0 = _mm256_loadu_ps(&v2_start[j]);
    _mm256_storeu_ps(&vout_start[row_offset + j],
                     _mm256_mul_ps(v1_vec, v2_0));

    __m256 v2_1 = _mm256_loadu_ps(&v2_start[j + AVX_STRIDE]);
    _mm256_storeu_ps(&vout_start[row_offset + j + AVX_STRIDE],
                     _mm256_mul_ps(v1_vec, v2_1));

    __m256 v2_2 = _mm256_loadu_ps(&v2_start[j + 2*AVX_STRIDE]);
    _mm256_storeu_ps(&vout_start[row_offset + j + 2*AVX_STRIDE],
                     _mm256_mul_ps(v1_vec, v2_2));

    __m256 v2_3 = _mm256_loadu_ps(&v2_start[j + 3*AVX_STRIDE]);
    _mm256_storeu_ps(&vout_start[row_offset + j + 3*AVX_STRIDE],
                     _mm256_mul_ps(v1_vec, v2_3));

    __m256 v2_4 = _mm256_loadu_ps(&v2_start[j + 4*AVX_STRIDE]);
    _mm256_storeu_ps(&vout_start[row_offset + j + 4*AVX_STRIDE],
                     _mm256_mul_ps(v1_vec, v2_4));

    __m256 v2_5 = _mm256_loadu_ps(&v2_start[j + 5*AVX_STRIDE]);
    _mm256_storeu_ps(&vout_start[row_offset + j + 5*AVX_STRIDE],
                     _mm256_mul_ps(v1_vec, v2_5));
}

for (; j < v2len - (AVX_STRIDE - 1); j += AVX_STRIDE) {
    __m256 v2_vec = _mm256_loadu_ps(&v2_start[j]);
    _mm256_storeu_ps(&vout_start[row_offset + j],
                     _mm256_mul_ps(v1_vec, v2_vec));
}

for (; j < v2len; j++) {
    vout_start[row_offset + j] = v1_val * v2_start[j];
}
```

As we can see in Snippet 2.A, we have unrolled the loop by a factor of 6, using 6 independent vector registers to store 8 floats at a time. This means that in a single loop iteration, 48 elements of each vector are handled. Figure 2.1 shows the effect of loop unrolling for different unroll factors. It doesn't make as big a difference as it does with the serial code, but it removes a sizable 600ns chunk from MVM time, and smaller effects for the other two. This, again, is expected

because MVM unrolling allows the functional pipelines to get to full capacity, whereas for VVM and MMA, unrolling only serves to reduce loop overhead. 6 unrolls is ideal once again.

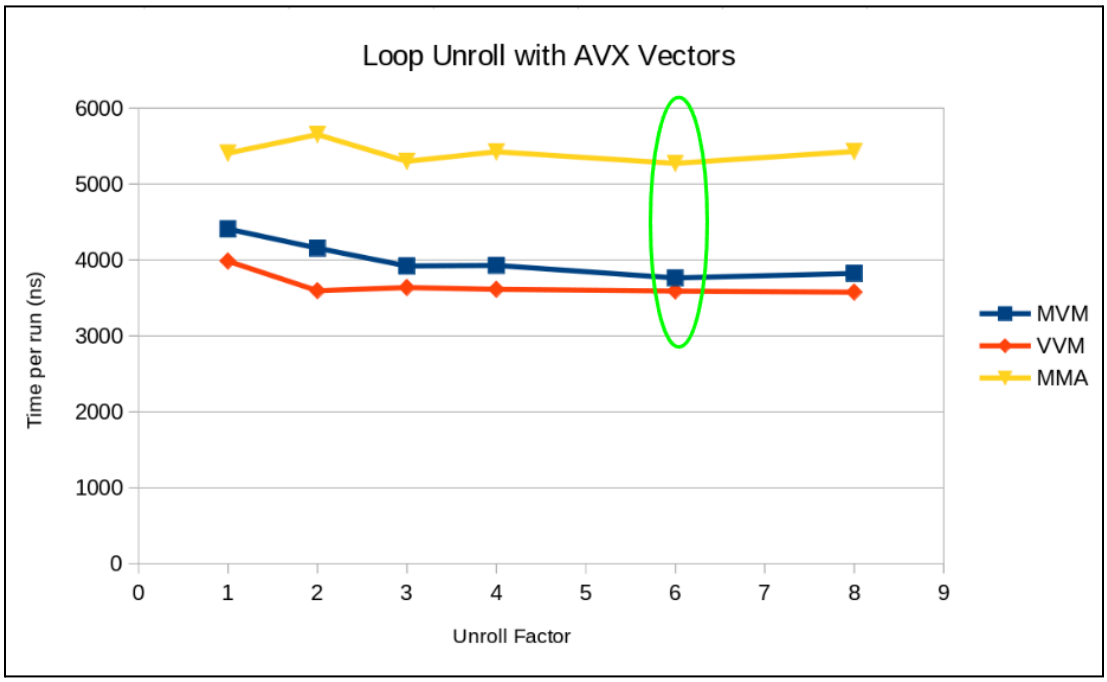

Figure 2.1 Effect of Loop Unrolling with Vectors

AVX Special Functions

AVX comes with some special functions that combine many of the simpler functions to do complex operations. In our project, two such functions were pertinent: _mm256_fmadd_ps, and _mm_hadd_ps. The first is a fused multiply-add, which performs vector multiplication and addition in one step without having to write back to an intermediate register. The second is a horizontal add operation, which takes 2 lanes and reduces them to 1. These functions were only relevant for MVM, as this is the only function that involves multiplication and addition (fused multiply-add), as well as accumulation (horizontal add). The fused multiply-add function was used to replace regular AVX multiply and add functions, and the horizontal add was used to replace a temp array stored in memory that was used to reduce the vector previously. In Snippet 2.B, 8 weights in ‘w0’ are multiplied by 8 activations in ‘a0’, and then added to 8 sums in ‘acc0’.

Snippet 2.B Example of how _mm256_fmadd_ps is used in matrix_vector_mult

```
__m256 w0 = _mm256_loadu_ps(&weights[row_offset + j]);
__m256 a0 = _mm256_loadu_ps(&lastLayerActivations[j]);
acc0 = _mm256_fmadd_ps(w0, a0, acc0);
```

In Snippet 2.C, the vector ‘sum128’ is reduced twice with _mm_hadd_ps to yield the sum of all the elements in the vector.

Snippet 2.C Example of how _mm_hadd_ps is used in matrix_vector_mult

```
sum128 = _mm_hadd_ps(sum128, sum128);
sum128 = _mm_hadd_ps(sum128, sum128);
```

In Figure 2.2, we can see the performance improvement of these functions for MVM. As we only changed MVM, only MVM shows an improvement, with a ~200ns reduction in time per run.

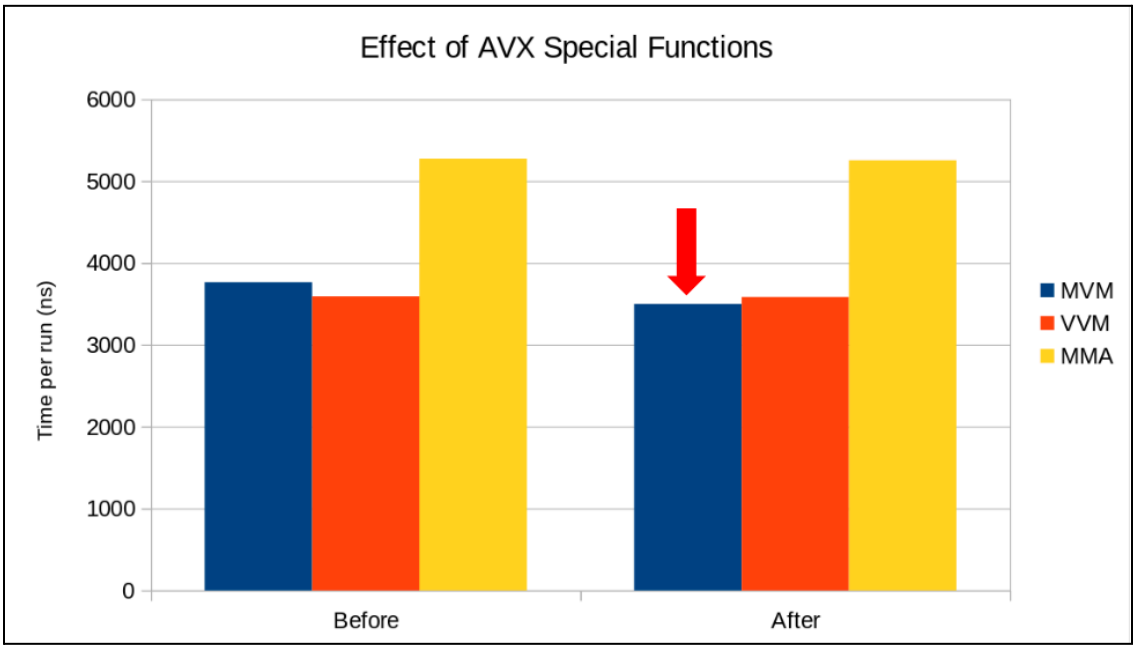

Figure 2.2 Effect of Special AVX Functions

3. Multicore Processing

Before describing the method for implementing multicore processing, it is important to acknowledge a constraint that is fundamental to the algorithm itself. This mini-batch stochastic gradient descent is inherently sequential at the batch level. Each batch produces a parameter update that the next batch depends on; in other words, the model learns and improves progressively.

Within a single batch, however, there is opportunity for parallelism. Each sample processed during training, from feedforward to backpropagation, is completely independent of other samples. The only dependency is that each sample's contribution to the gradient must be accumulated before the next batch begins processing. So then, we divide the batch evenly among the number of specified threads. This required a bit of refactoring, especially the separate scratch space and private gradient accumulators to avoid race conditions and cache thrashing between threads. The threads will then rejoin and share their work among global accumulators so that the parameters can be updated. The pseudocode below highlights the new changes.

Snippet 3.A Pseudo-code describing new changes for Multithreading Parallelization

```
// INIT
allocate scratch[NUM_THREADS]
allocate thread_grad_sums[NUM_THREADS]
allocate global_grad_sum

// TRAINING
for epoch = 1 .. E
    for each batch of size B

        #pragma omp parallel for schedule(static)
        for each sample in batch
            tid = omp_get_thread_num()
            feedforward(scratch[tid])
            backpropagation(scratch[tid], label)
            thread_grad_sums[tid] += scratch[tid].sample_grads
        end

        for t = 0 .. NUM_THREADS
            global_grad_sum += thread_grad_sums[t]
            zero(thread_grad_sums[t])
        end

        update_parameters(global_grad_sum)
    end
end
```

Results

When implementing this multithreading strategy, we parameterized the number of threads to test for optimal efficiency. Figure 3.0 displays the runtime over various thread counts. Both the total time and training time are displayed, though they follow an identical pattern further supporting the target kernels of optimization.

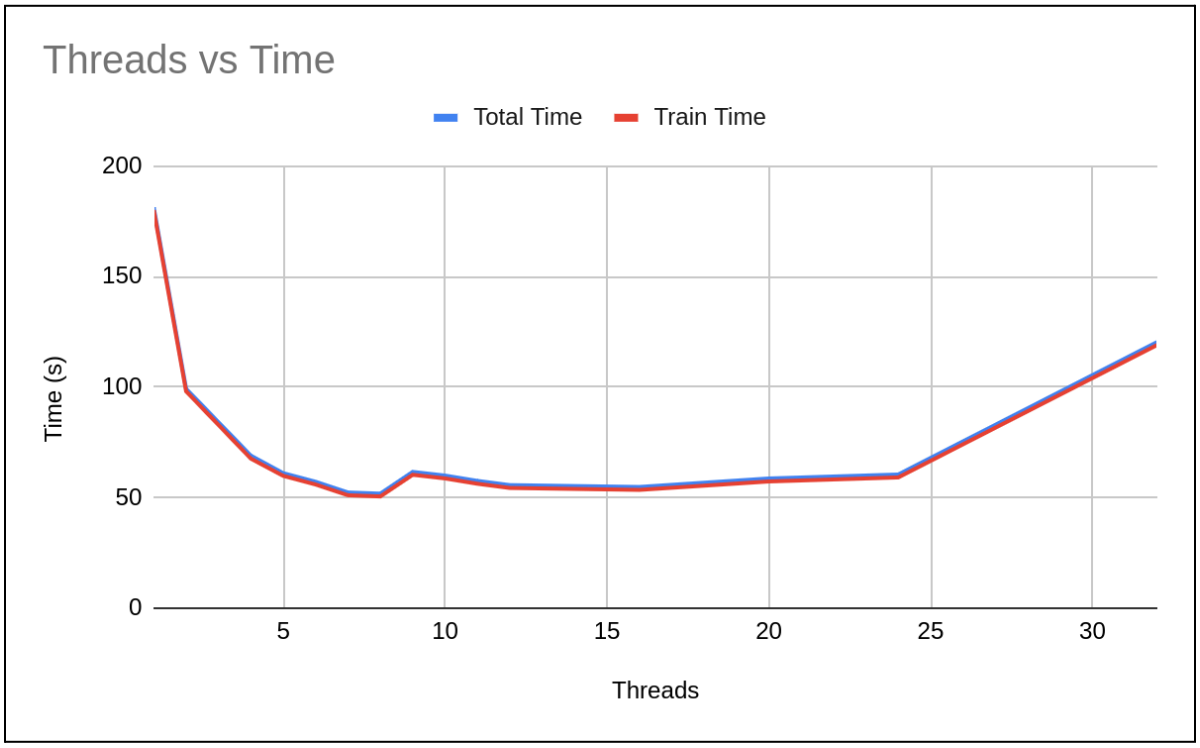

Figure 3.0 Effect of Varying Thread Count on Time

As shown, the initial introduction of additional threads provides huge returns for the time of our program. Relatively quickly, however, these returns diminish after about 5 threads. There are three primary factors for this. The first is that this method still goes through the training process serially for each image, this is a limitation and is fixed in later iterations of parallel processing. Additionally, with a batch size of 100 and a relatively small overall model, we are spinning up separate threads with very little work to do. Lastly, our algorithm is memory-bound, which means that parallelization is not limited by the speed of each core, but instead the saturation of buses. There is also potential for false sharing, as memory is accessed by block. In some cases, the samples will not evenly distribute to the threads. Our implementation handles this, but it means that some threads will be waiting while others have finished. The machine that we ran all of our tests on has a hybrid core architecture, meaning that beyond assigning to the 8 performance cores, we will see worse performance on additional efficiency cores. Also, beyond the 24 physical cores, hyperthreading kicks in, saturating memory and degrading performance.

4. All Together 1.0!

Combining the best version of each of the optimization approaches, **we were able to reduce our total runtime by more than 50X** down from 195.1802s to 3.8933s. Targeting the 'inner' portions of our code, we applied the vectorization methods to the kernel functions. Targeting the 'outer' portions of our code, we applied multithreading by partitioning the batches. Verifying that we did not change the core functionality of our model, we maintained an accuracy of 96.57% throughout.

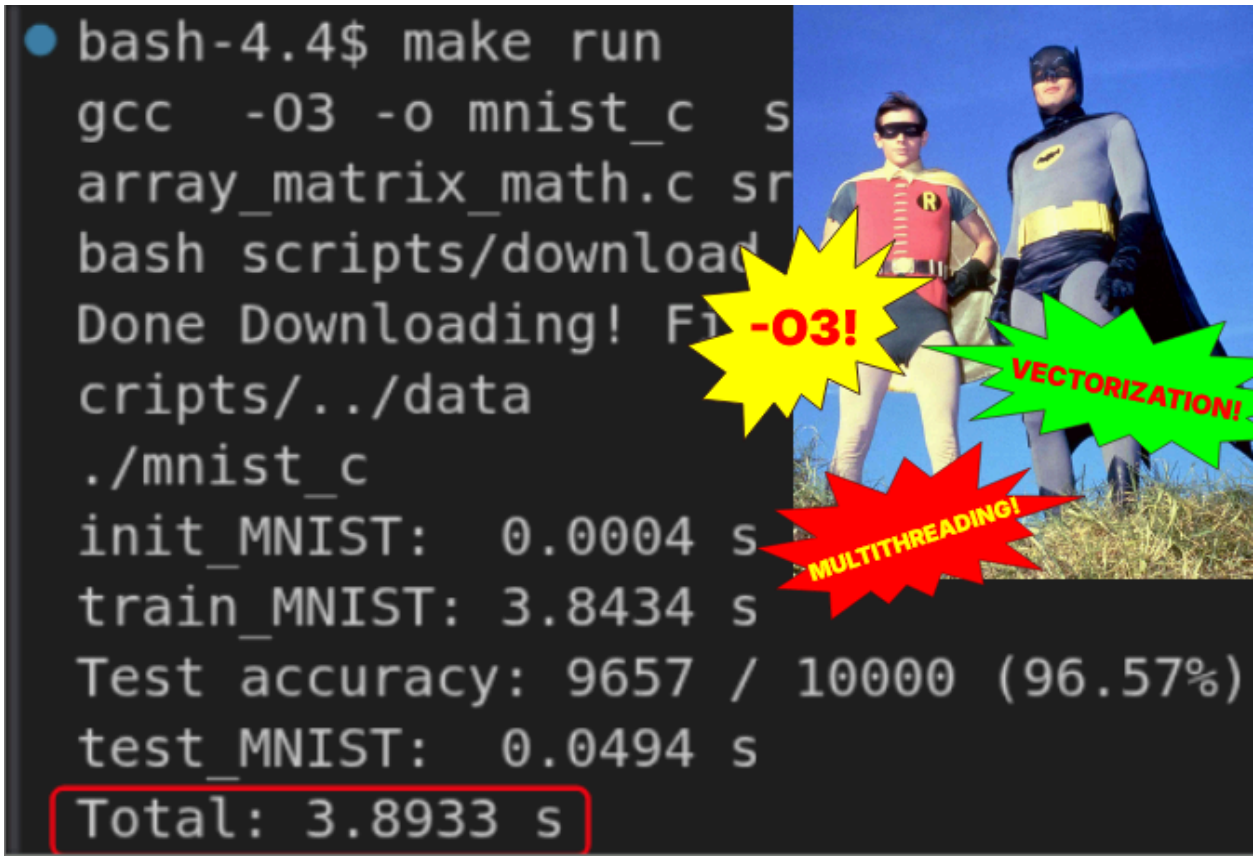

Figure 4.0 Terminal Results for 1.0

5. Memory Access Improvements

After putting everything together, we started looking into further optimizations. In our 3.9s "All Together 1.0" version, we only replaced and optimized the MVM, VVM, and MMA that operated on the matrices and vectors involving the input layer (784) and first hidden layer (100). An easy next step was to simply swap out the unoptimized MVM, VVM, and MMA functions for the other layer transitions with the optimized versions. This got us down to 2.5s.

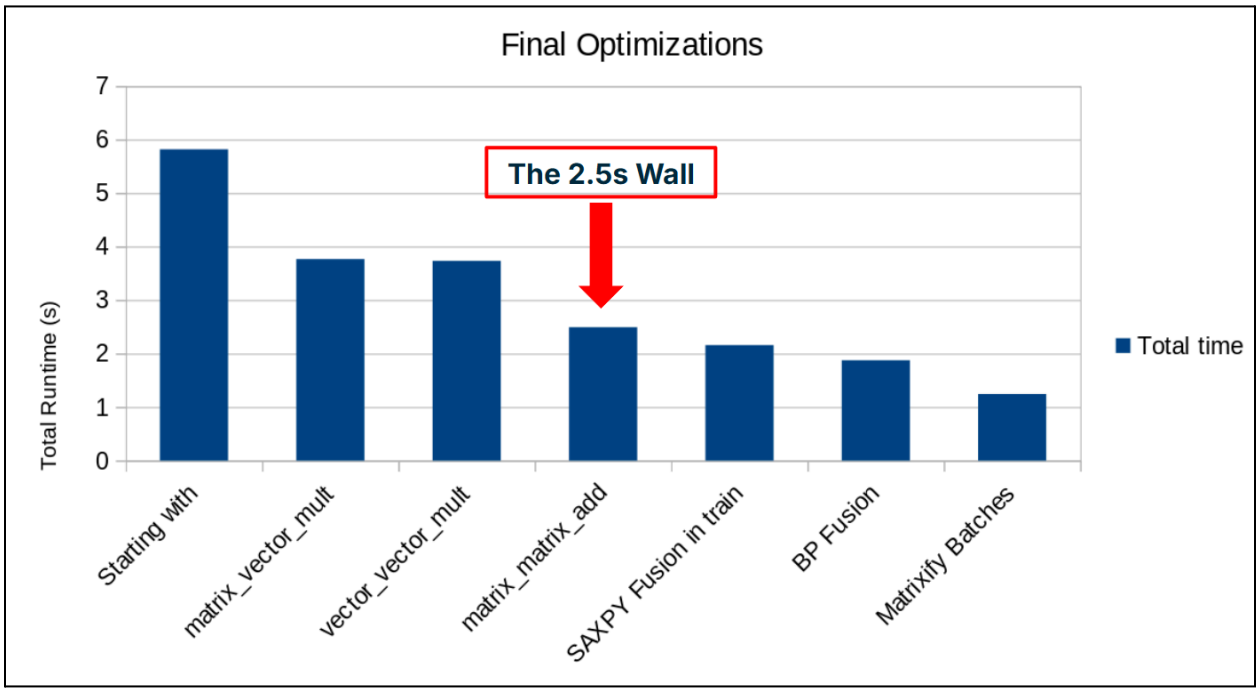

Figure 5.0 Final Optimizations

Next, we experimented with optimizing other functions in the program, such as vector_vector_add, matrix_transpose, and matrix_scalar_multiply. Unfortunately, all attempts to vectorize and loop unroll proved futile, as our total runtime sat constant at 2.5s. All the other functions in our model were simply too memory-bound and had stride-1 memory access patterns which could not be improved with blocking or loop interchange work.

We had exhausted the compute optimizations and now sat at a memory-bound wall of 2.5s. What to do? We racked our brains and consulted LLMs to help figure out how we might improve our number of memory accesses. We found two main techniques:

1. Fusing together functions to reduce memory accesses
2. Changing our one-image-at-a-time training method to all-images-at-once

Being strapped for time, we fell back on using LLMs to write the vast majority of the code in this section. We still did our due diligence in understanding the core principles in these optimizations.

Function Fusion

Almost as cool as nuclear fusion, function fusion is taking multiple functions and merging them into one. Terrible for modularity, good for memory accesses. In our code, we implemented two main function fusions: SAXPY and alterations to the Backpropagation function.

SAXPY is Single-precision A times X plus Y. One of the key components of our training algorithm is to accumulate the gradients of all weights and biases for a given batch, divide them by the number of images to get the average, multiply them by the Learning Rate, multiply them by -1 so that a matrix-matrix add can become a matrix-matrix subtract, and then add them to the weight and bias data structures. These 4 operations, 3 mults and 1 add, are shown in Snippet 5.A.

Snippet 5.A Example of the unoptimized three multiplies and the add operation in train_MNIST

```
data_t reciprocalBatchSize = 1.0 / BATCH_SIZE;

// Average weight gradients, scale by learning rate, negate, and apply update to
weights
matrix_scalar_mult(H0_W_grad_sum, reciprocalBatchSize, H0_W_grad_sum);
matrix_scalar_mult(H0_W_grad_sum, (data_t)LEARN_RATE,  H0_W_grad_sum);
matrix_scalar_mult(H0_W_grad_sum, -1.0,                H0_W_grad_sum);
matrix_matrix_add(H0_W_grad_sum, H0_W, H0_W);
```

For each of these multiplies and adds, the function loads the data from memory, operates on it, and stores it back to memory. 4 x (1 load + 1 store) x 100x784 x 4 bytes = ~2.5MB of data transfer to update the H0_W weight matrix. Here we apply function fusion. We technically only need to load all of this data once and store all of it once, so why don't we just stuff all of these operations into one function? Snippet 5.B shows what that looks like:

Snippet 5.B Example of the optimized SAXPY operation in train_MNIST

```
data_t scale = -(data_t)LEARN_RATE / BATCH_SIZE;

kernel_matrix_saxpy(H0_W_grad_sum, scale, H0_W);
kernel_matrix_saxpy(H1_W_grad_sum, scale, H1_W);
kernel_matrix_saxpy(L_W_grad_sum,  scale, L_W);

kernel_vector_saxpy(H0_B_grad_sum, scale, H0_B);
kernel_vector_saxpy(H1_B_grad_sum, scale, H1_B);
kernel_vector_saxpy(L_B_grad_sum,  scale, L_B);
```

The function kernel_matrix_saxpy takes a scalar, multiplies it by each element of the matrix, then adds this result to another matrix, and then stores it at the memory address of the other matrix. We optimized this function with vectorization and loop unrolling as well. 4x fewer

memory accesses means ~600KB of data transfer instead, much lower. Figure 5.0 shows a 13.4% total time reduction.

Next, we made some major changes to our backpropagation algorithm. Prior to this optimization, backprop used, among other smaller matrices, a 100x784 matrix (314KB) to store the temporary gradient for each image that was passed through. There were 3 memory accesses to that matrix 12.5 times on average per thread, giving a total memory transfer of 12.5 x 3 x 314KB = ~11.8 MB. What we realized is that this matrix can be completely bypassed by accumulating the image gradients directly into the gradient sum. This required a pointer to the gradient sum arrays to be passed into the backprop function, but resulted in significantly fewer memory transfers. Figure 5.0 shows an improvement of 13.1% as a result of this change.

Image Combine

The second major technique we implemented to reduce memory accesses was processing multiple images simultaneously in training. As previously discussed in Section 3, we used batches of 100 images, and we split these up among 8 threads, with each thread being responsible for accumulating the gradients of 12 or 13 images. Originally, each thread processed its assigned samples sequentially, running feedforward and backpropagation on one image at a time before accumulating its contribution into the thread-local gradient sum. See Snippet 5.C. This required loading each activation vector, weight matrix, and bias vector 12.5 times on average from memory during the lifetime of a thread. If, instead, we packed all 12 or 13 images into a matrix and turned all the MVMs into MMMs, and other vector functions into their corresponding matrix functions, then we would only need to load all these massive data structures once during the thread's lifetime. See Snippet 5.D. This shift, although simple in principle, required a massive overhaul of our suite of functions, which not only needed to be created, but also optimized with vectorization, blocking/tiling (now applicable with MMM), and loop unrolling. The work was expedited considerably by LLMs.

Snippet 5.C Before implementing all-at-once batch, notice the for loop

```
#pragma omp parallel for schedule(static)
for (int j = i; j < i + BATCH_SIZE; j++) {
  int tid = omp_get_thread_num();
  SampleScratch *s  = &scratch[tid];
  ThreadGradSum *ts = &thread_sums[tid];

  copyImageToInput(train_data, s->IN, indices[j]);
  int num = train_data->nums[indices[j]];

  feedforward(s);
  backprop(s, ts, num);   // accumulates directly into ts
}
```

Snippet 5.D After implementing all-at-once batch, notice the absence of the for loop

```
#pragma omp parallel
{
  int tid      = omp_get_thread_num();
  int start    = (tid * BATCH_SIZE) / NUM_THREADS;
  int end      = ((tid + 1) * BATCH_SIZE) / NUM_THREADS;
  int actual_S = end - start;

  BatchScratch  *s  = &scratch[tid];
  ThreadGradSum *ts = &thread_sums[tid];

  /* Pack images: indices[i+start .. i+start+actual_S-1] → IN_batch */
  copyImagesToInputBatch(train_data, s->IN_batch,
                         &indices[i + start], actual_S);

  /* Pack labels */
  for (int k = 0; k < actual_S; k++)
    s->labels[k] = train_data->nums[indices[i + start + k]];

  feedforward_batch(s, actual_S);
  backprop_batch(s, ts, actual_S);
}
```

With these major changes introduced, we were able to reduce the total time by another 34%, bringing us down to 1.24 seconds.

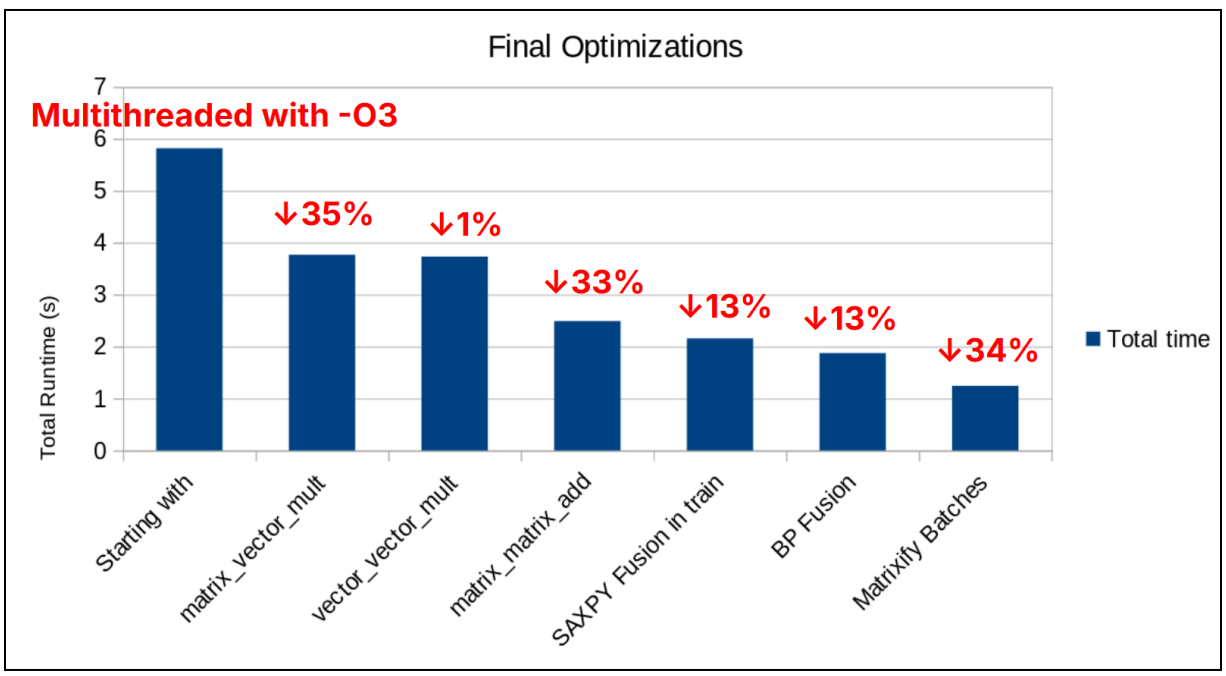

Figure 5.1 Final Optimizations with Percent Decreases*

*One caveat: The small improvement in VVM is not consistent with earlier improvements due to -O3 being set

6. All Together 2.0!!!

Results

At last, we have arrived! Maintaining a testing accuracy of 96.57%, we have achieved a final performance boost of **156X** speedup from 195.1802s to 1.2449s.

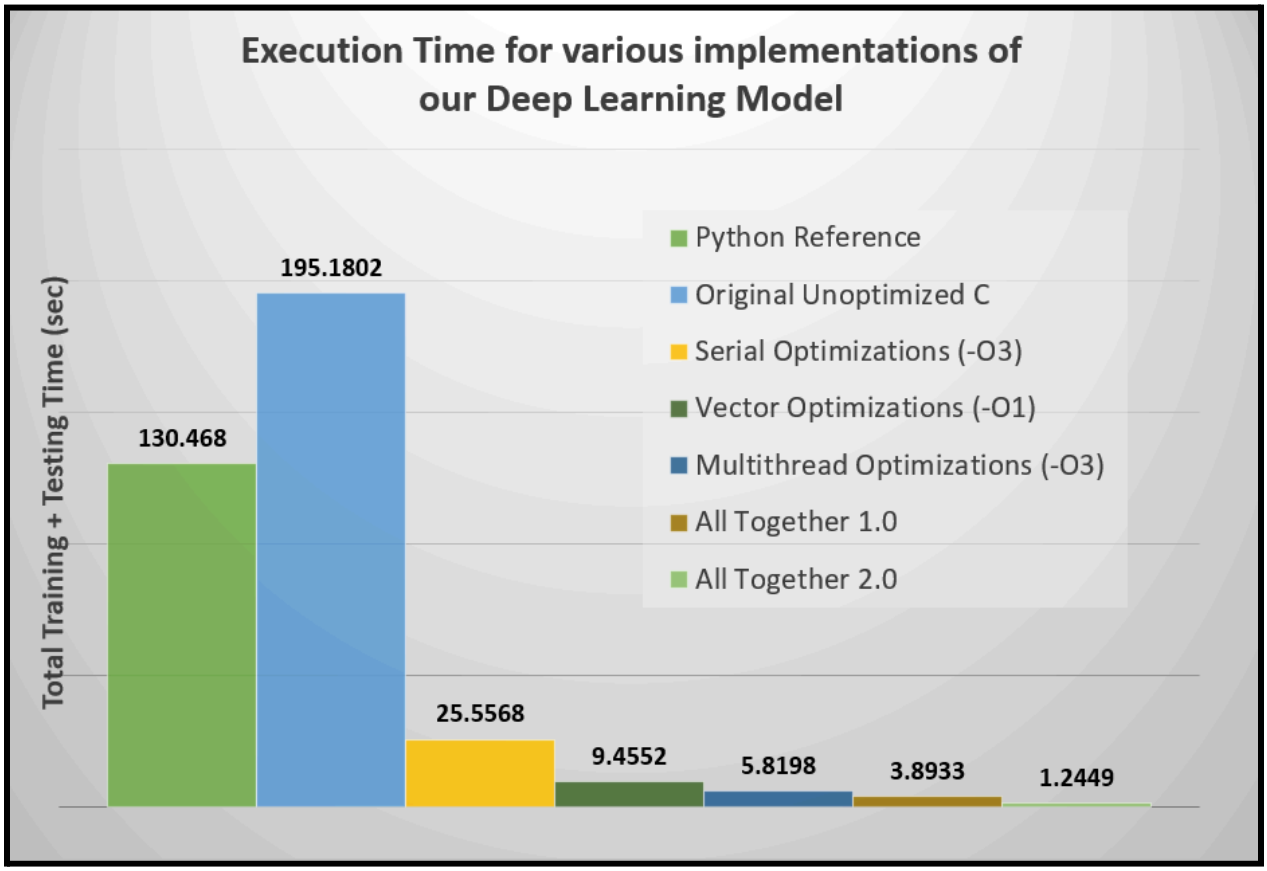

Figure 6.0 Final Results

Snippet 6.A Our final run with all our most optimized code - 1.2449 seconds!

```
bash-4.4$ make run
gcc  -O3 -o optimized  src/mnist_loader.c src/main.c src/model.c src/array_matri
x_funcs.c src/array_matrix_math.c src/kernels.c -lm -fopenmp -mavx2 -mfma
bash scripts/download_data.sh
Done Downloading! Files are in /ad/eng/users/c/r/crossd/EC527/FINAL_PROJECT/EC52
7_HighPerformanceProgramming_Project/optimized/scripts/../data
./optimized
init_MNIST:  0.0006 s
train_MNIST: 1.2127 s
Test accuracy: 9657 / 10000 (96.57%)
test_MNIST:  0.0315 s
Total: 1.2449 s
```

A useful metric of comparison is the runtime of our original reference Python code from which we built our model. Using identical hyperparameters, and even a smaller training data size of 50,000 images, the results are shown below.

Snippet 6.B Python reference code time.

```
real    2m10.468s
user    15m25.259s
sys     38m29.882s
```

It's worth noting that the user time is ~7X the real time, implying that this reference version is actually parallelized in the background. Applying our high-performance programming techniques, we were able to surpass this baseline threshold substantially.

Conclusion

We have a few key takeaways about Optimizing Deep Learning from this project.

1. In deep learning models like ours, where the number of nodes decreases by a large factor in each subsequent layer (in our case, ~8), the vast majority of the compute time is spent between the first and second layers.
2. Expanding on this, minimizing memory accesses is super important for memory-bound systems like this. In our case, it was extremely valuable to reduce the number of memory accesses to the massive weight matrices associated with the first-to-second-layer transition.
3. As we were warned when setting out on this project, building a deep learning model from scratch that has enough parameters to exhibit the latencies and slowness as well as the potential optimizations associated with matrices that only fit in L3 and RAM memory is difficult. In our case, only the weight matrices associated with the transition from the input layer to the first hidden layer were too large to fit in L1 cache. All other data structures fit easily in L1 cache, making our functions overwhelmingly memory-bound. If our model had been significantly larger, our optimizations for other functions (like vector_vector_add and matrix_transpose) likely would have been worth it.
4. One of our goals when approaching this project was to learn the inner workings of deep learning models. Building this neural network from the ground up forced us to work intimately with the math behind artificial intelligence. The value of this was emphasized when it came time to apply our high-performance programming techniques. Throughout our course assignments, the main difficulties we encountered were in the actual implementation of a given technique. In the context of this project, however, we began to realize that the actual difficulty is being creative enough to figure out what optimizations might work.

References

[1] Google Gemini, “Neural Network Construction Illustration,” AI-generated image, April 2026.
[2] Sanderson, G., “Neural Networks,” *3Blue1Brown*, YouTube, 2017. https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi.
[3] Nielsen, M. A., “Neural Networks and Deep Learning,” *Determination Press*, 2015. http://neuralnetworksanddeeplearning.com/
[4] Nielsen, M. A., “neural-networks-and-deep-learning,” GitHub Repository, 2015. https://github.com/mnielsen/neural-networks-and-deep-learning.

Property	Value
OS	AlmaLinux
CPU	Intel Core i9-14900
Architecture	x86_64
P-Cores (Performance)	8 (hyperthreaded, 2 threads each)
E-Cores (Efficiency)	16 (1 thread each)
Total Logical CPUs	32
Base / Boost Clock	2.0 GHz / 5.8 GHz
L1 Cache (d/i)	48K / 32K per core
L2 Cache	2048K per core
L3 Cache	36 MB (shared)
NUMA Nodes	1

Note: P-core hyperthreading yields 16 logical threads from 8 physical cores. E-cores do not support hyperthreading.

Figure R.0 Machine Characteristics